

AUTOMATION PORTFOLIO — TECHNICAL RECORD

SOPHON FINANCE SYSTEMS

AI-Driven Finance & Accounting Automation | github.com/sophonfinance-wq/finance-automation-portfolio

READ-ONLY VALIDATION ENGINE

A deterministic, strictly read-only rules-based validation engine that runs a registry of independent checks over finished financial workbooks (.xlsx) and their JSON exports, flagging pre-sign-off defects without ever mutating the audited file.

Report No.	SFS-ENG-005	Package	validation_engine
System	Read-Only Validation Engine	Data	Fictional / seeded
Run	python run.py	Status	Public — portfolio record

1.0 OVERVIEW AND SCENARIO

A reviewer is about to sign off on a batch of near-identical surplus workbooks, any of which may hide a defect that survives eyeball review. Running the engine over its seeded fictional corpus surfaces exactly these: Birchwood Op Co's trial balance fails to tie out (debit 885 vs credit 875, a 10 plug); Maple Fund LP's closing surplus was quietly hardcoded to 900 instead of the formula =B3+B4+B5; and Demo Holdings LLC's published JSON export reports closing_surplus 1725 while the workbook computes 1700. Each is returned as a numbered, cell-located exception with an overall verdict of FAIL, so nothing defective reaches sign-off.

2.0 WHAT THE SYSTEM AUTOMATES

- Confirms totals that must be formula-driven (Surplus-Detail!B6, Summary!B2) actually hold formulas rather than quietly hardcoded literals (expected_formula → FAIL)
- Verifies the trial balance ties out by summing debit and credit columns and comparing within tolerance (debit_credit_balance → FAIL)
- Scans every non-empty cell for stale review notes, internal, or tooling words — TODO, FIXME, TBD, 'reviewer decision', 'do not ship', model/tool names (forbidden_text → FLAG)
- Checks lineage direction — evidence cells must be raw literal inputs and detail-driver cells must be formulas; either inversion is flagged (lineage_direction → FLAG)
- Detects leftover draft-only MIN/MAX cap logic that should have been replaced with final logic (cap_logic_leftover → FLAG)
- Cross-checks the sibling JSON export's closing_surplus against the workbook's resolved closing surplus (json_tieout → FAIL)
- Generates a seeded, fully fictional workbook corpus (one clean baseline plus exactly one defect per rule) and emits both a markdown and a JSON validation report with per-finding status and location

3.0 OPERATING PROCEDURE

- 1 Generate (or reuse) the seeded fictional corpus into ./samples — one clean workbook plus one workbook per planted defect, each with a sibling .json export

- 2 Load each .xlsx read-only twice: once with data_only=False to inspect formulas, once with data_only=True for cached values; auto-attach any sibling <stem>.json
- 3 Run the ordered registry of six independent checks over each workbook, each yielding zero or more Findings tagged PASS / FAIL / FLAG with a Sheet!Cell or json:<key> location
- 4 Roll a workbook's findings into a verdict: any FAIL $\Rightarrow$ FAIL, otherwise any FLAG $\Rightarrow$ REVIEW, otherwise PASS
- 5 Roll all workbook verdicts into a single overall verdict as the worst-of across the run
- 6 Serialize results into validation_report.md (human-readable tables) and validation_report.json (machine-readable, with tool/version/registry/counts)
- 7 Close both workbook handles in a finally block, leaving every audited file byte-identical
- 8 Return a process exit code encoding the verdict (0 PASS · 1 REVIEW · 2 FAIL · 3 usage/IO error) for pipeline gating

4.0 CONTROLS AND SAFEGUARDS

- Strictly read-only: workbooks are opened with read_only=True and never written back, so the engine is structurally incapable of introducing a defect — it can only report
- Determinism: synthetic figures come from the stdlib random module under a fixed seed (20240614), and findings are emitted in a stable registry order, so the corpus and every finding are reproducible
- Severity separation with worst-of roll-up: hard FAILs (hardcoded total, out-of-balance, JSON mismatch) are distinguished from soft FLAGs (stale text, inverted lineage, leftover cap logic), preserving a human-review gate rather than blanket auto-fail
- Independent cross-source tie-out: the published JSON export is reconciled against the workbook it was derived from, catching divergence between the source and the distributed artifact
- Deterministic exit codes (0/1/2/3) provide an unambiguous, automatable control gate for CI or pre-sign-off pipelines
- Extensible registry: each rule is an isolated (WorkbookContext) -> list[Finding] function registered via a decorator, keeping checks declarative and independently testable

5.0 VERIFICATION AND TESTING

Correctness is proven by an automated pytest suite that the README reports as 1,314 passing. The tests assert that every planted defect is caught by its corresponding rule (one per rule) with the expected FAIL/FLAG status and a usable location, that the clean workbook passes with no FAIL or FLAG, and that both report formats are well-formed. Read-only behavior is verified by SHA-256 hashing every corpus file before and after a full validation run and asserting byte-for-byte equality, while determinism is verified by regenerating the corpus and re-running validation to identical findings. The CLI further encodes the verdict as a process exit code for external gating.

6.0 KEY FACTS

Attribute	Value
Validation rules in registry	6 (expected_formula, debit_credit_balance, forbidden_text, lineage_direction, cap_logic_leftover, json_tieout)
Fictional workbooks in corpus	7 (1 clean baseline + 6, one planted defect per rule)

Automated tests passing (per README)	1,314
Exit codes	0 PASS · 1 REVIEW · 2 FAIL · 3 usage/IO error
Fixed RNG seed	20240614
Package version	1.0.0

This record documents a self-contained system in the Sophon Finance Systems automation portfolio. All data is fictional and seeded; no employer or client workpaper, entity, methodology, path, or figure is reproduced. Prepared 14 July 2026.